Lecture – Cache memory, part 1

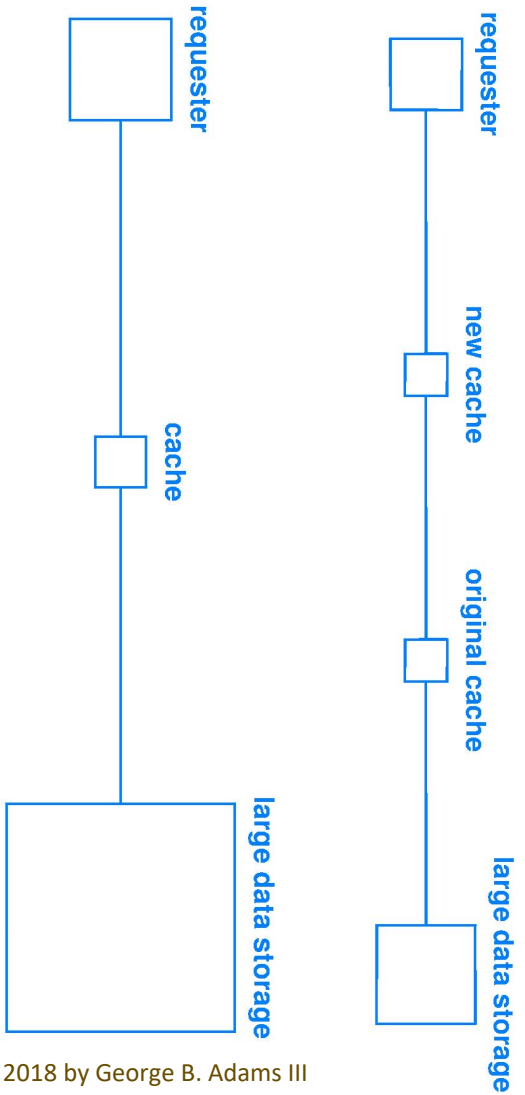
Cache – A small, hidden storage place for valuables.

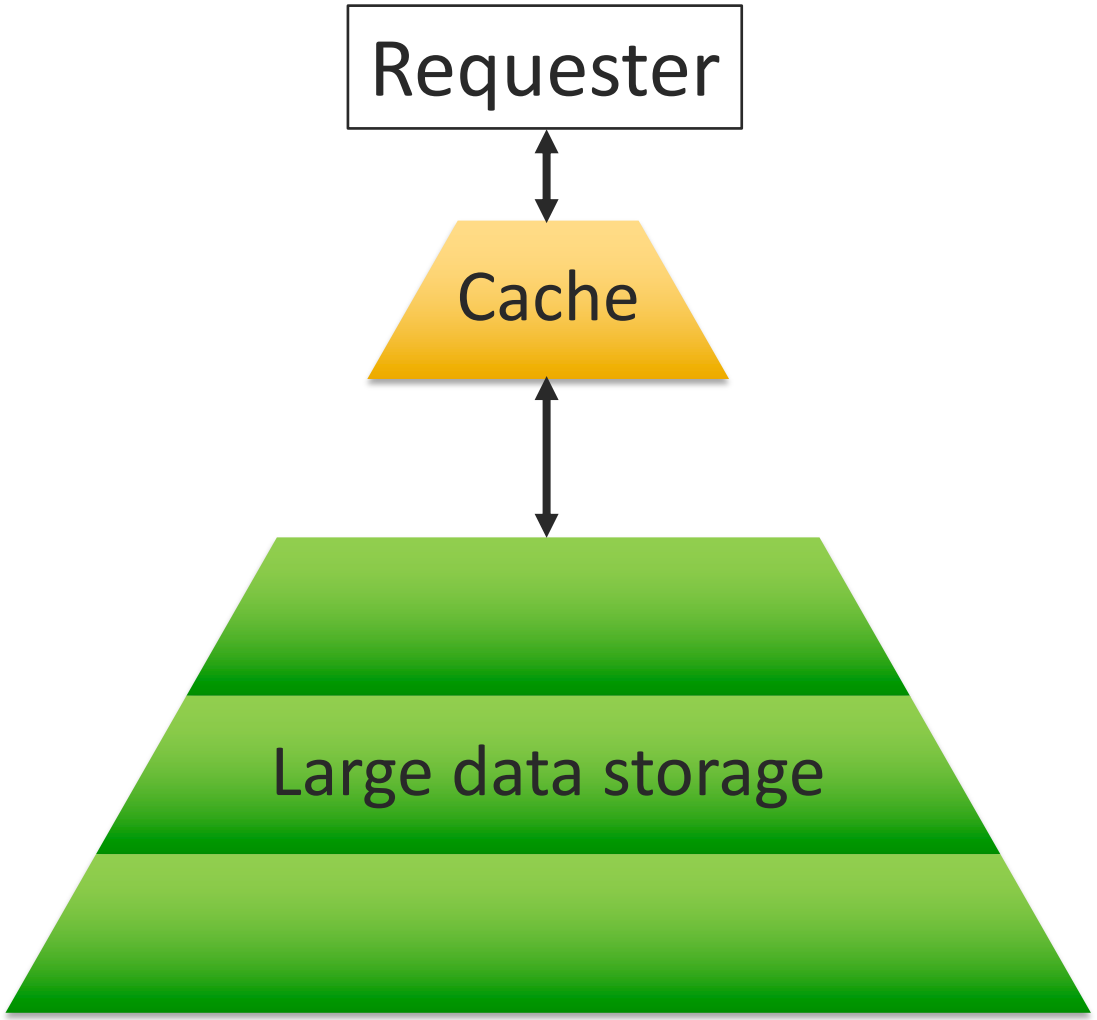
What is Modified Harvard and 3 Rules?

- Read chapter 5
- Learning objectives
  - Explain the Modified Harvard Architecture and the I-cache and D-cache design
  - Remember the Three Rules of the Hierarchy

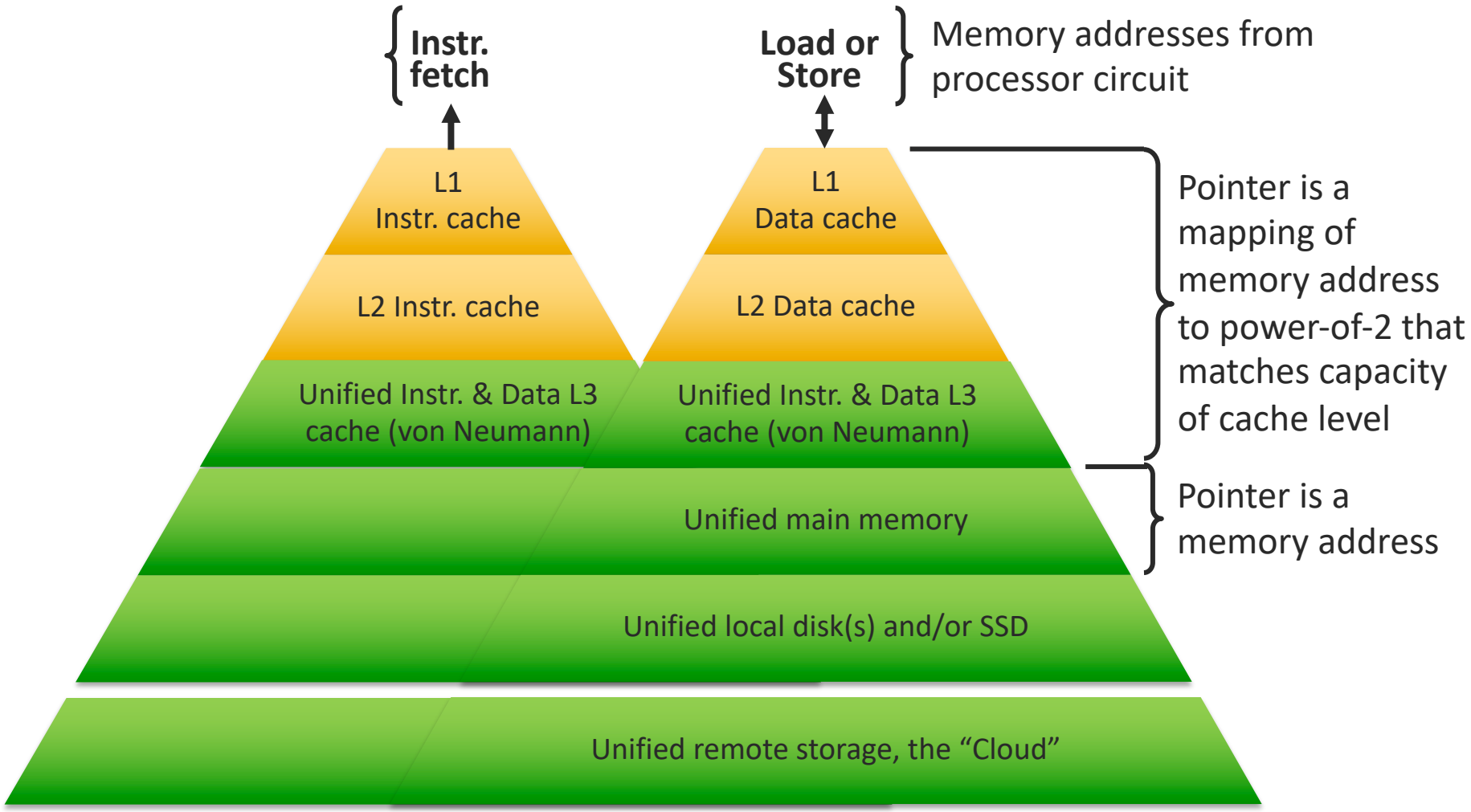
Requestor, cache, memory hierarchy

- Blue figures are universal





Memory hierarchy – addressing



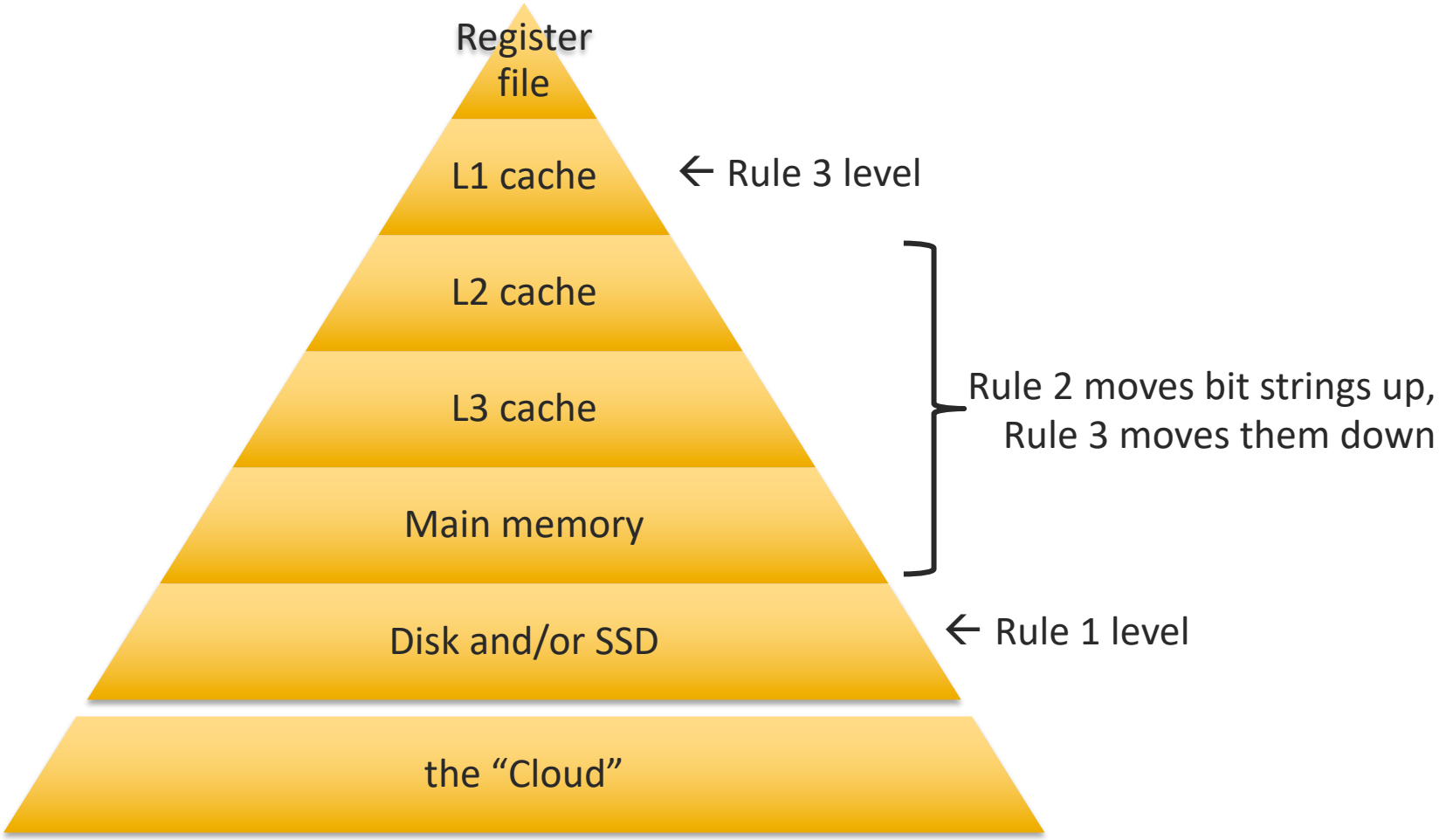
Three Rules of the memory hierarchy

1. The lowest level in the hierarchy is the ultimate repository for all information long-term

2. A level above is provided a copy of information from the level below when the level above cannot satisfy a processor memory access

3. When the processor stores (writes) information into the top level, that change must eventually propagate down the levels to maintain Rule 1. *(Propagation is transparent to user programs.)*

Memory hierarchy – by the Three Rules

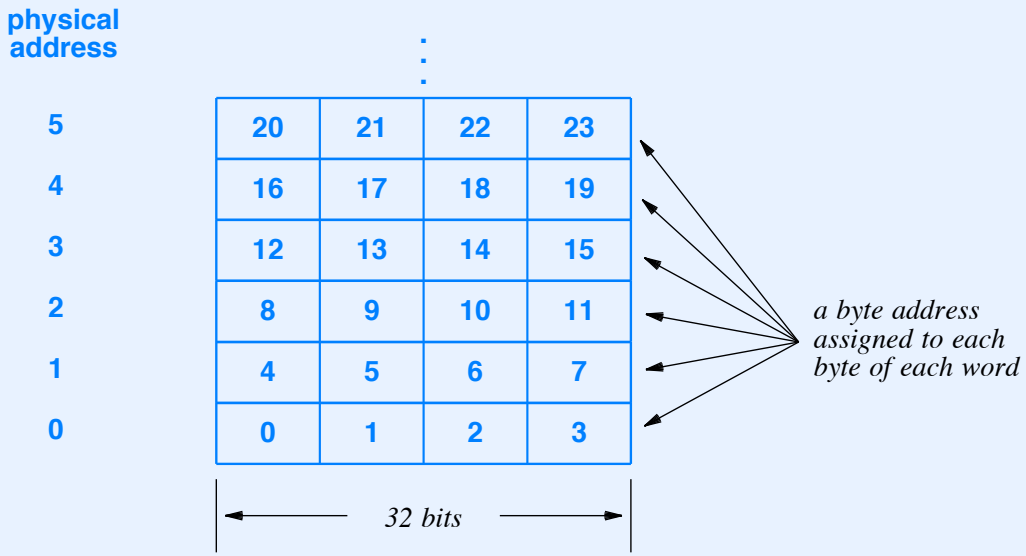


What is direct mapped cache organization?

- Read chapter 5
- Learning objectives
  - Explain the circuit for and operation of a direct mapped cache

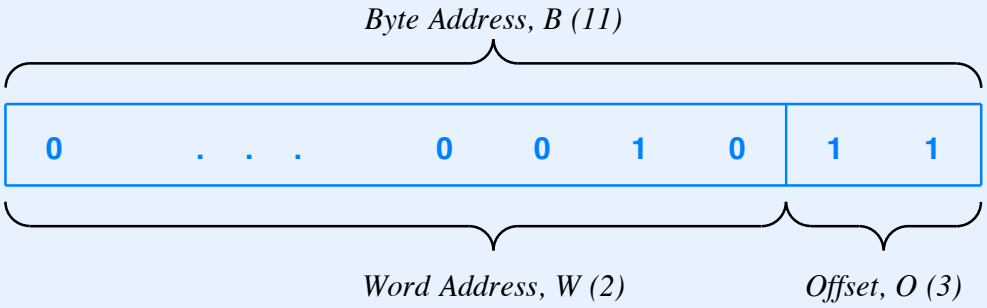
Recall Example Of Address Translation

- Assume physical memory is organized into 32-bit words
- Programmer views memory as an array of bytes
- We think of each byte has having an address 0 through *N–1*
- Each physical word corresponds to 4 byte addresses



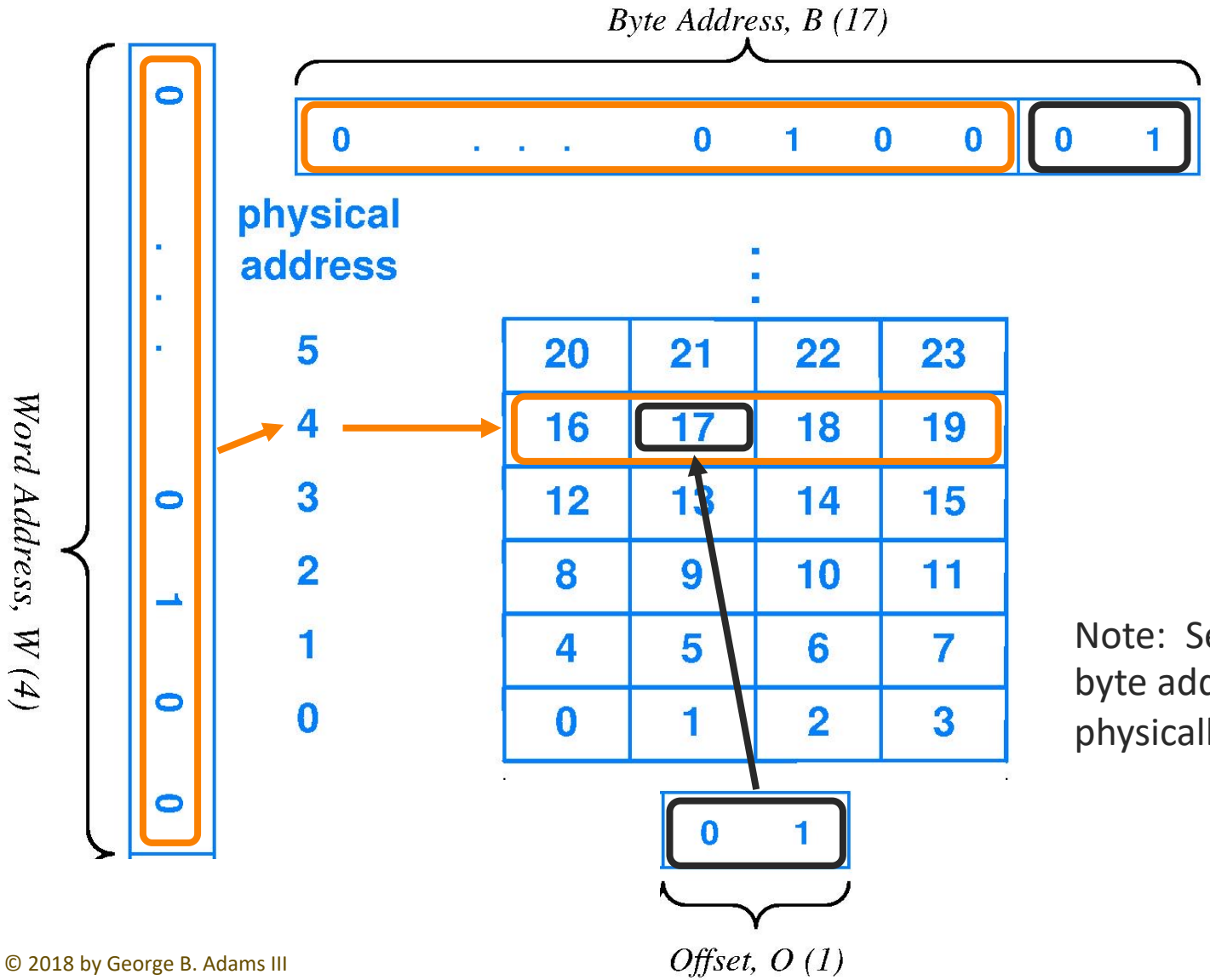
Fast, Efficient Translation

- Think binary and choose word size *N* to be a power of 2
- Avoids arithmetic calculations, especially division and remainder
- Word address computed by extracting high-order bits
- Offset computed by extracting low-order bits
- Example: byte 11 with *N* equal to 4 bytes per word



2D: Word address is one dimension, offset, another

2D example: byte address = 17

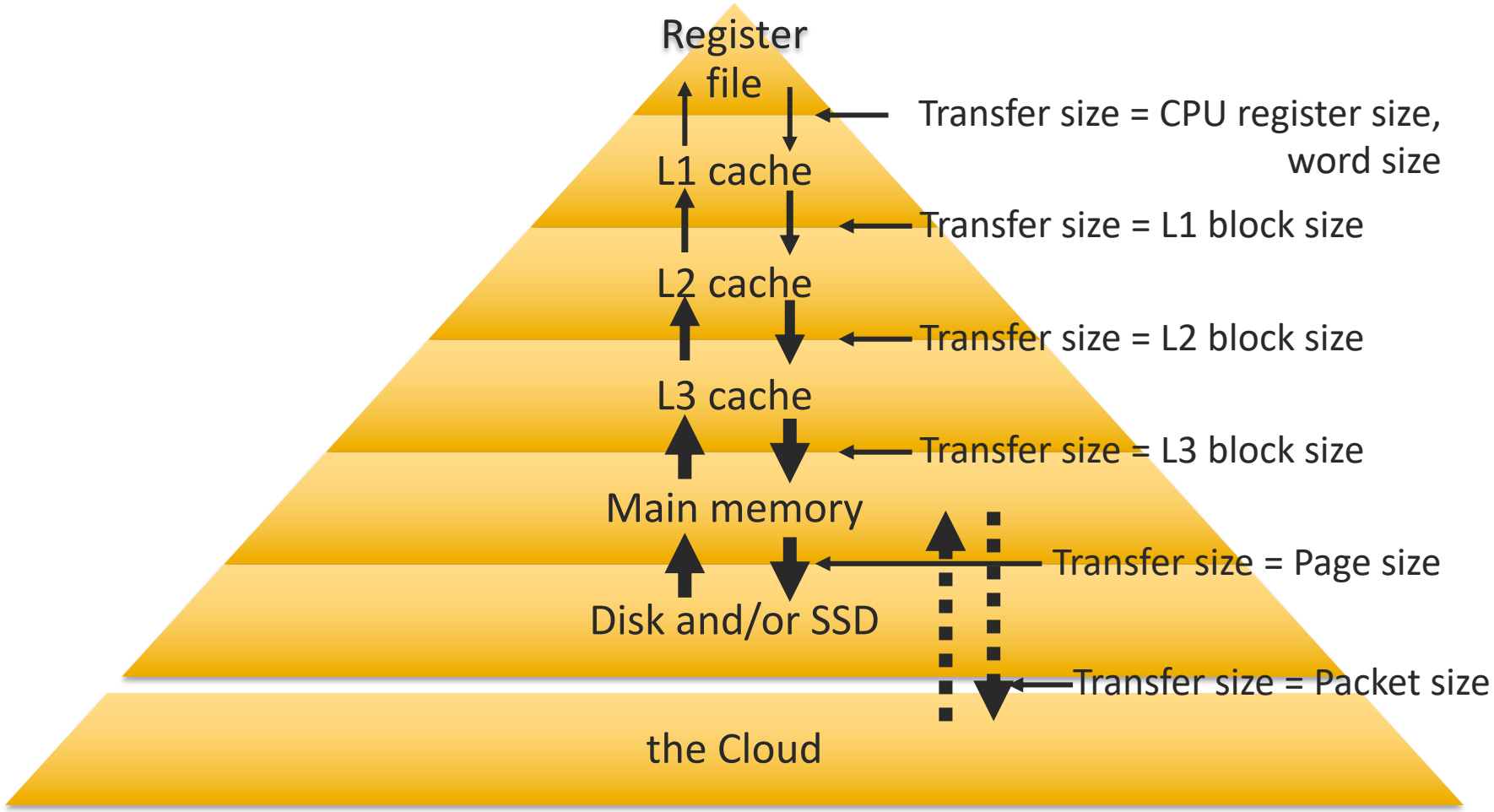


Note: Sequential byte addresses are physically close

Block, generalization of the word

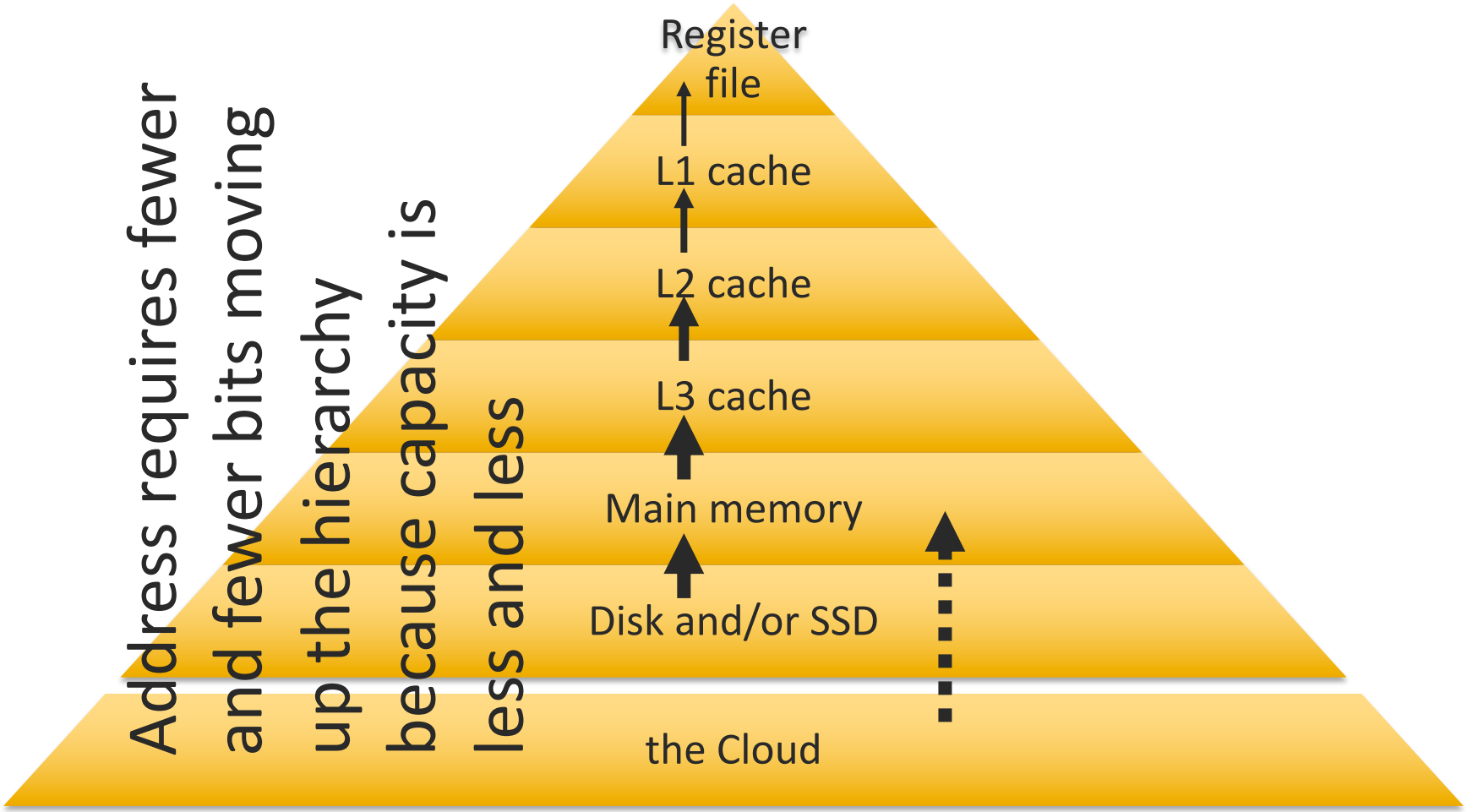
- Bit string that moves between adjacent memory hierarchy levels is called a block, *generically*
- Blocks have various common names
- Block size is a power of 2 multiple of general purpose register size (processor word size)
- Block contains sequential word addresses

Memory hierarchy – block transfer size



Arrows are thinner at higher levels to show block size decreases as memory technology latency decreases

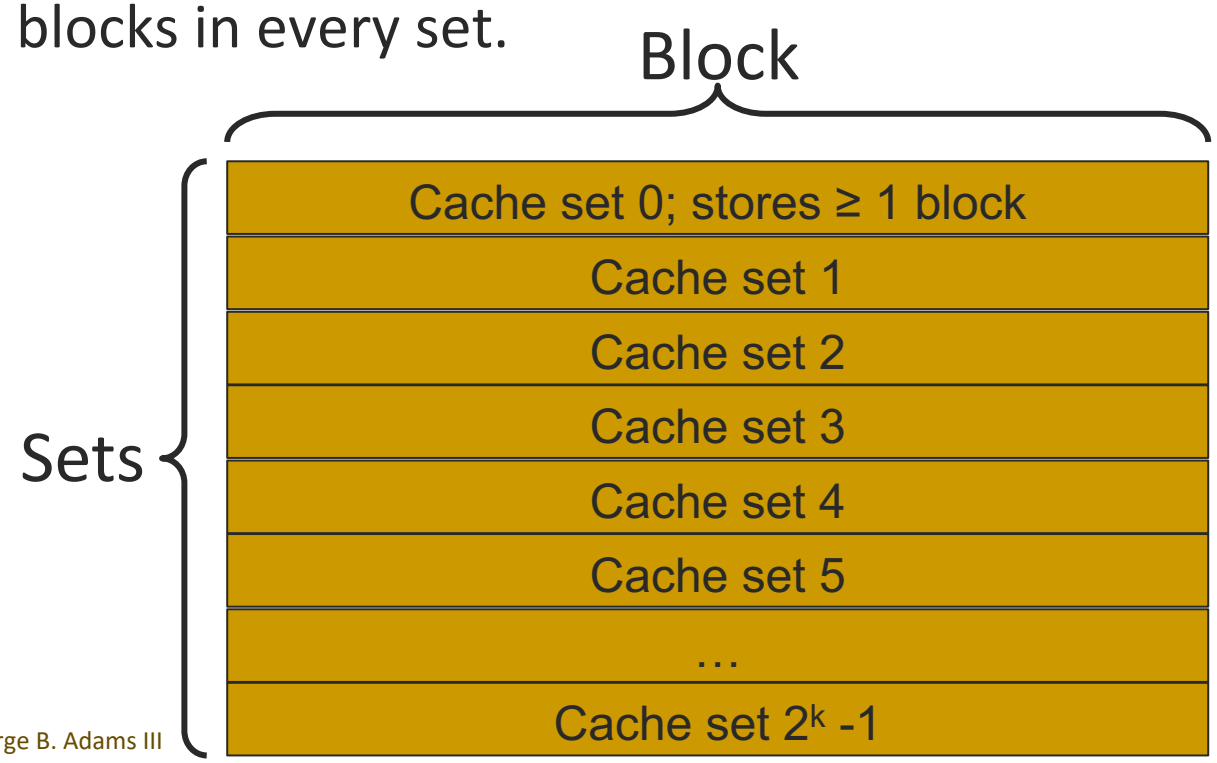
Memory hierarchy – copy up on access



Copy/write arrows are thinner at higher levels to show block size decreases as latency decreases

Cache addressing generalizes word addressing

- Instead of storing words, the stored items are blocks
  - A block contains a smallish power of 2 number of words
- A pointer for a cache points to location called a set
  - Each set stores one **or more** blocks! Same number of blocks in every set.

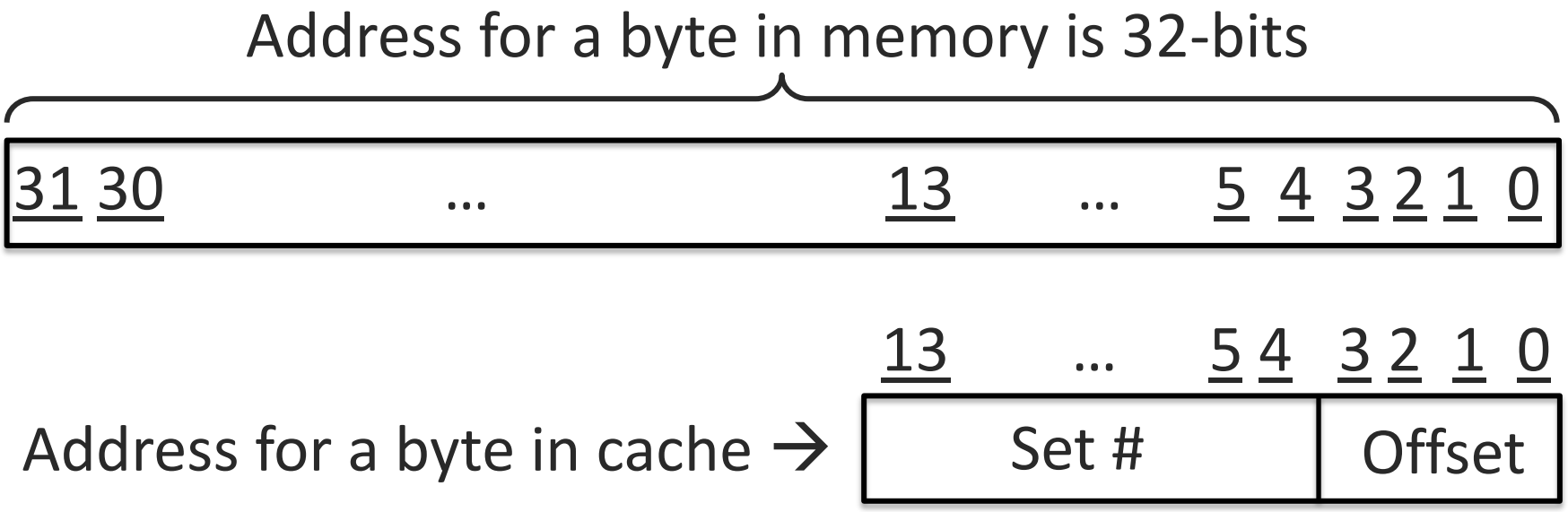


Cache addresses are small (few bits)

- SRAM is fast, hot, and expensive
  - ∴ number of bytes in cache << bytes in working memory
  - ∴ number of address bits needed for cache < number of bits in a working (main) mem address
- Translation of main memory address to a cache address MUST be super fast per Amdahl's Law because **memory access is very frequent**

Fastest function – Use just enough low-order bits

- Example: 32-bit byte-address main memory, 32-bit words, and a cache of 1024 sets and 4-word blocks
- Compute cache address by taking low-order bits: *propagation delay = zero gate delays, i.e, fast*



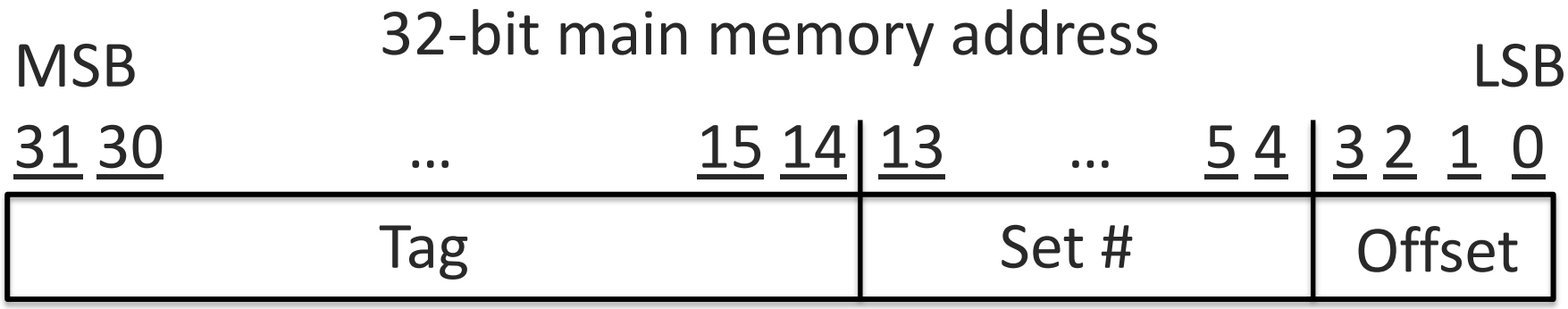
Problem with using low-order bits

- Example memory contains 2^32 bytes
- Cache address points to 2^14 bytes
- Bytes copied into cache from main memory
- When point to a byte in cache using 14-bit address, then *Which one of the 2^18 possible 32-bit addresses is the one from which the byte was copied?*
- Only way to know: store the 18 high-order address bits along with the byte

Representation used for main memory address

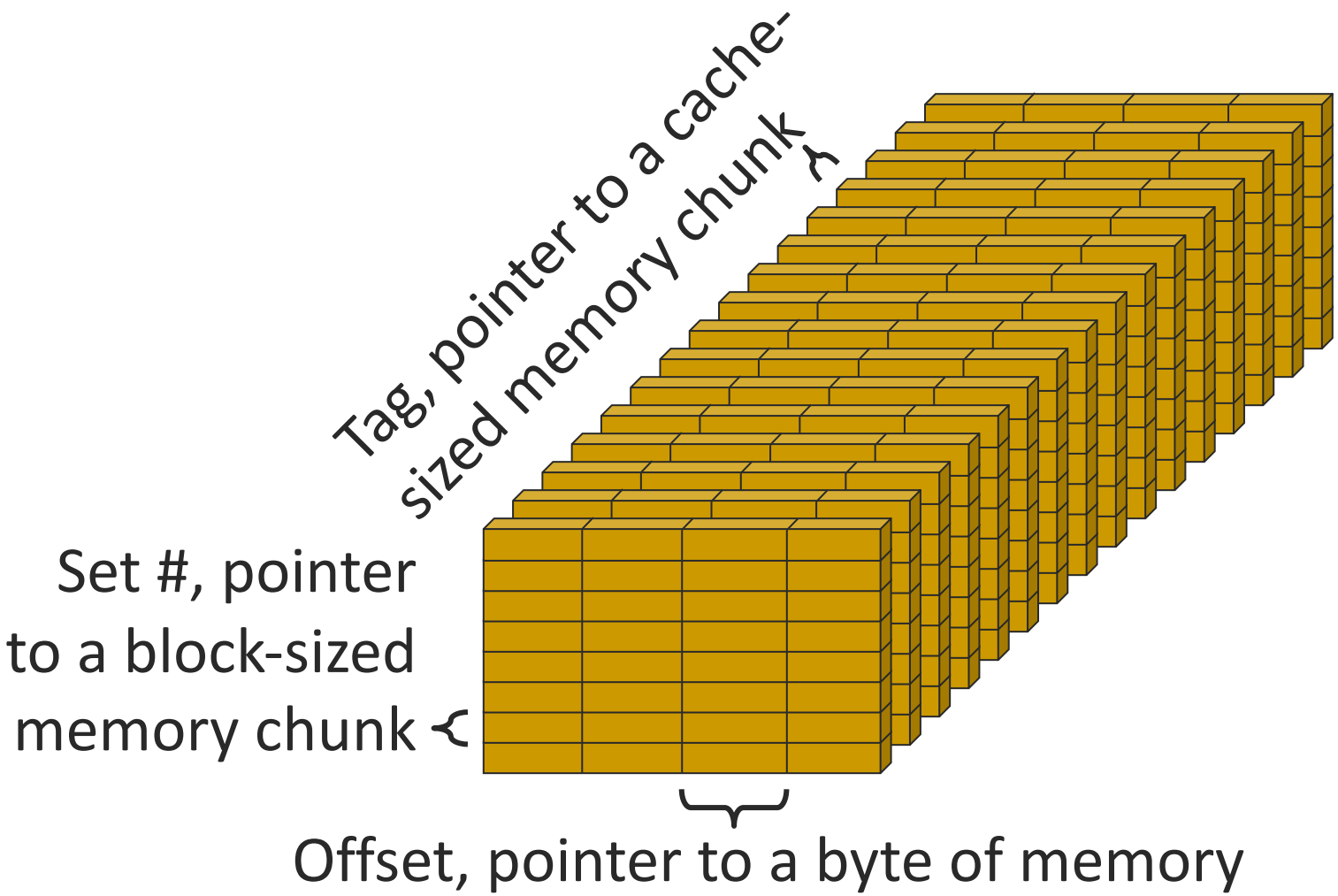
- Offset – points to a byte within a block
- Set # – points to a storage area for block(s) in the cache
- Tag – points to a cache-sized group of consecutive bytes in main memory
- Number of bits in each field is a design choice

- Example



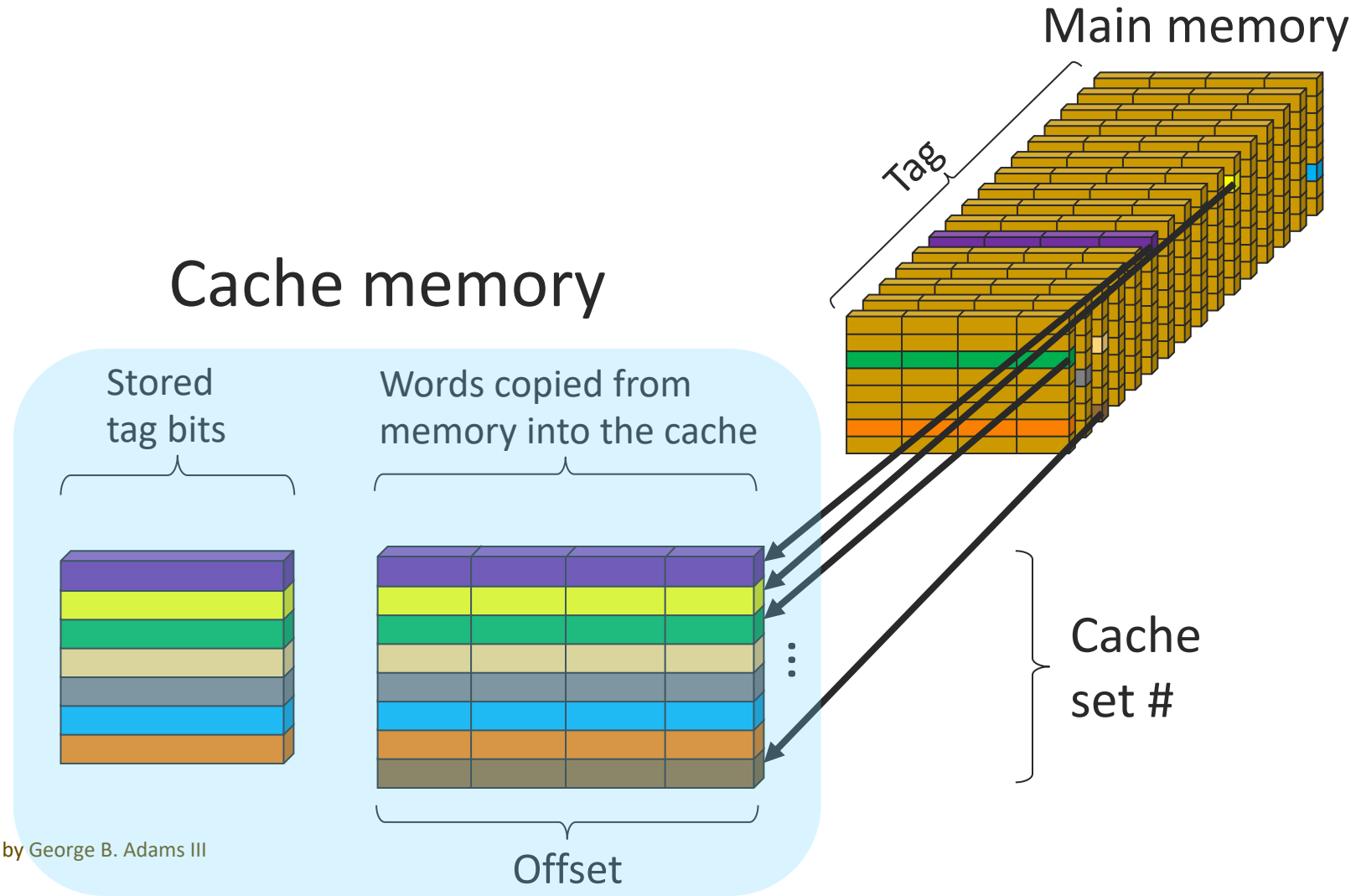
Physical memory now a 3D abstraction

- Tag, Set #, Offset: memory address from MSB to LSB



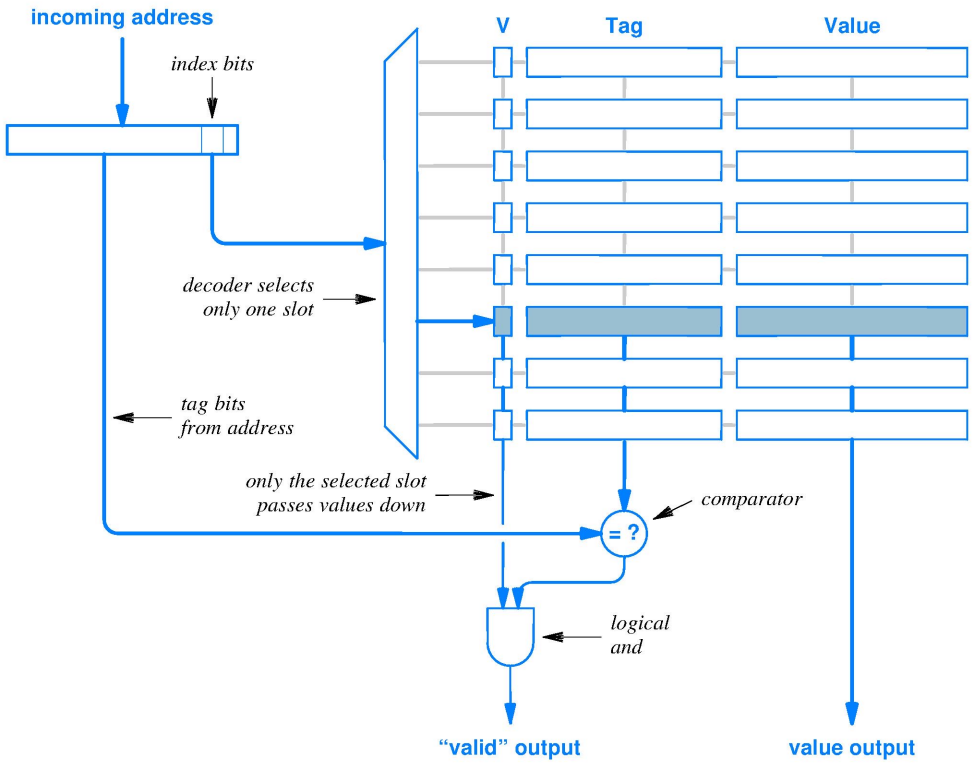
Direct-mapped cache

- Direct-mapped means there is 1 block in each set



Find word in cache given an aligned byte address

- "Index bits" represent the Set #
- Offset field ignored in this abstract diagram
- V is the valid bit, provides Enable functionality per block
- If incoming tag field matches stored tag of a block in the set pointed to by the index and Valid=True then cache HIT



Response to byte address from processor

- Hit – this hierarchy level has a copy of the word
  Response:
  - As necessary, pass block containing the word up the hierarchy, until block is in L1 cache
  - Because processor ONLY communicates with L1 caches, process the Fetch/LOAD/STORE using L1
- Miss – this level does not have a copy of the word
  Response:
  - Wait for some lower level in the hierarchy to HIT
  - Rule 1 of the Hierarchy guarantees a HIT, eventually
- Cancel request – a faster level above has *already HIT*
  - Needed if byte address is sent to multiple hierarchy levels simultaneously

Read algorithm for direct-mapped cache

- Given: byte address, ***A***
- Function: read word at address ***A***
- Method:
  - Extract tag ***T***, index ***I***, and offset ***O*** from byte address ***A*** /* group wires */
    if (tag stored at ***I*** == ***T*** && Valid_bit == true) { /* cache HIT */
      Use ***I*** to select block and ***O*** to select word; Deliver word to CPU;
    }
    else { /* cache Miss */
      /* hardware always updates cache content if Miss */
      Find block containing address ***A*** from *fastest hierarchy level* below;
      Write block into cache at index ***I ;*** /* overwrite existing block */
      tag at index ***I*** = ***T*** ;
      Valid_bit at index ***I*** = True;
      Use ***I*** to select block and ***O*** to select word; Deliver word to CPU;
    }

Caching and multidimensional arrays

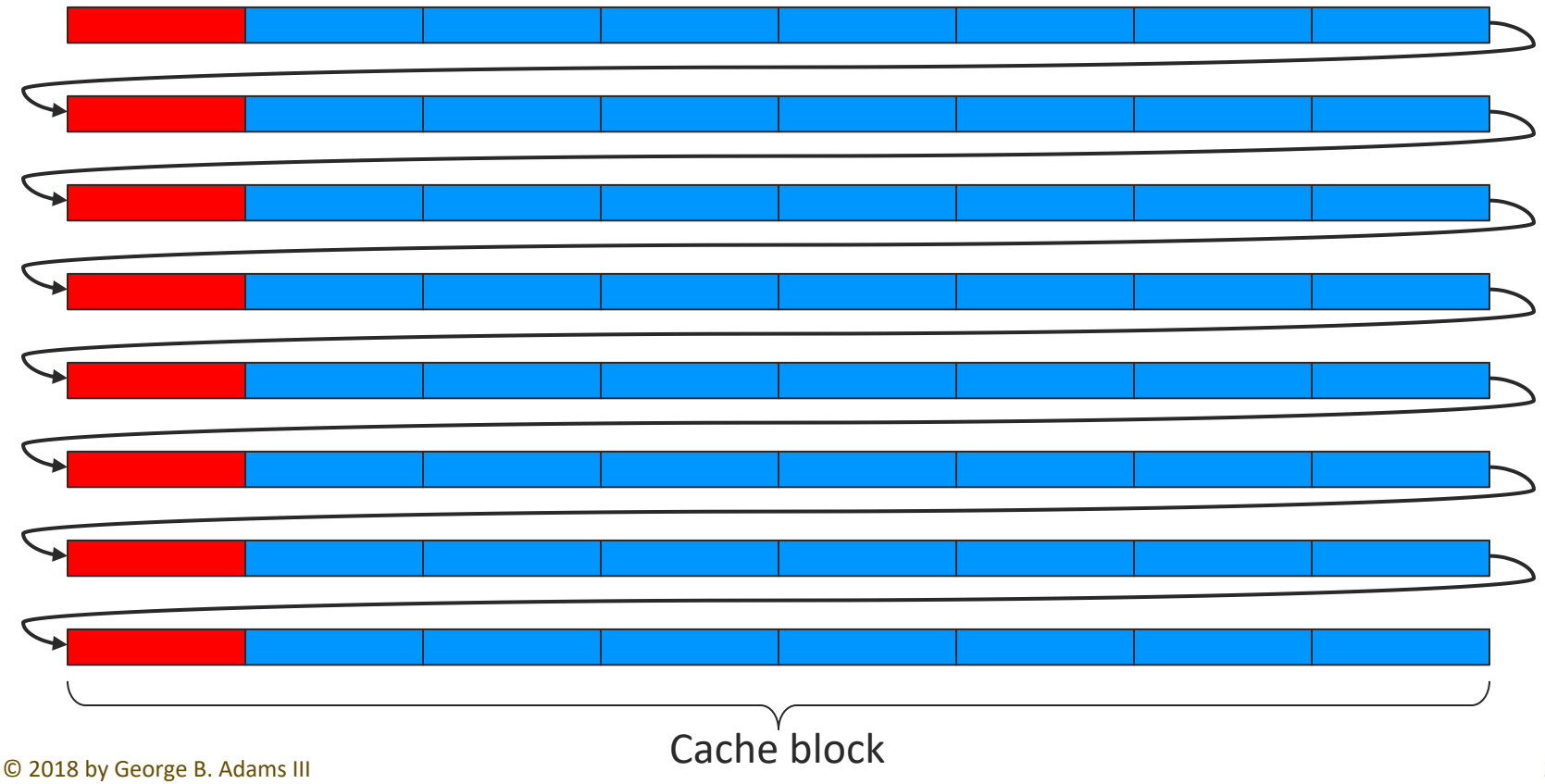
- Learning objectives
  - Understand that HLLs make a choice of how to store multidimensional arrays in memory
  - Understand the interaction of storage order, programmed access order, and cache hit ratio

Caching and multidimensional arrays

- Memory address space is 1-dimensional
- Arrays are N-dimensional
- How map N dimensions, say i, j, k, …, of array A[ ] onto 1 dimension? Where to store each element?
  - Row Major order
    A[0,0,…,0] followed by A[0,0,…,1], A[0,0,…,2], …
    rightmost dimension is sequential in computer memory
  - Column Major order
    leftmost array dimension is sequential in memory
- Programs that sequentially access sequentially stored array elements benefit the most from cache

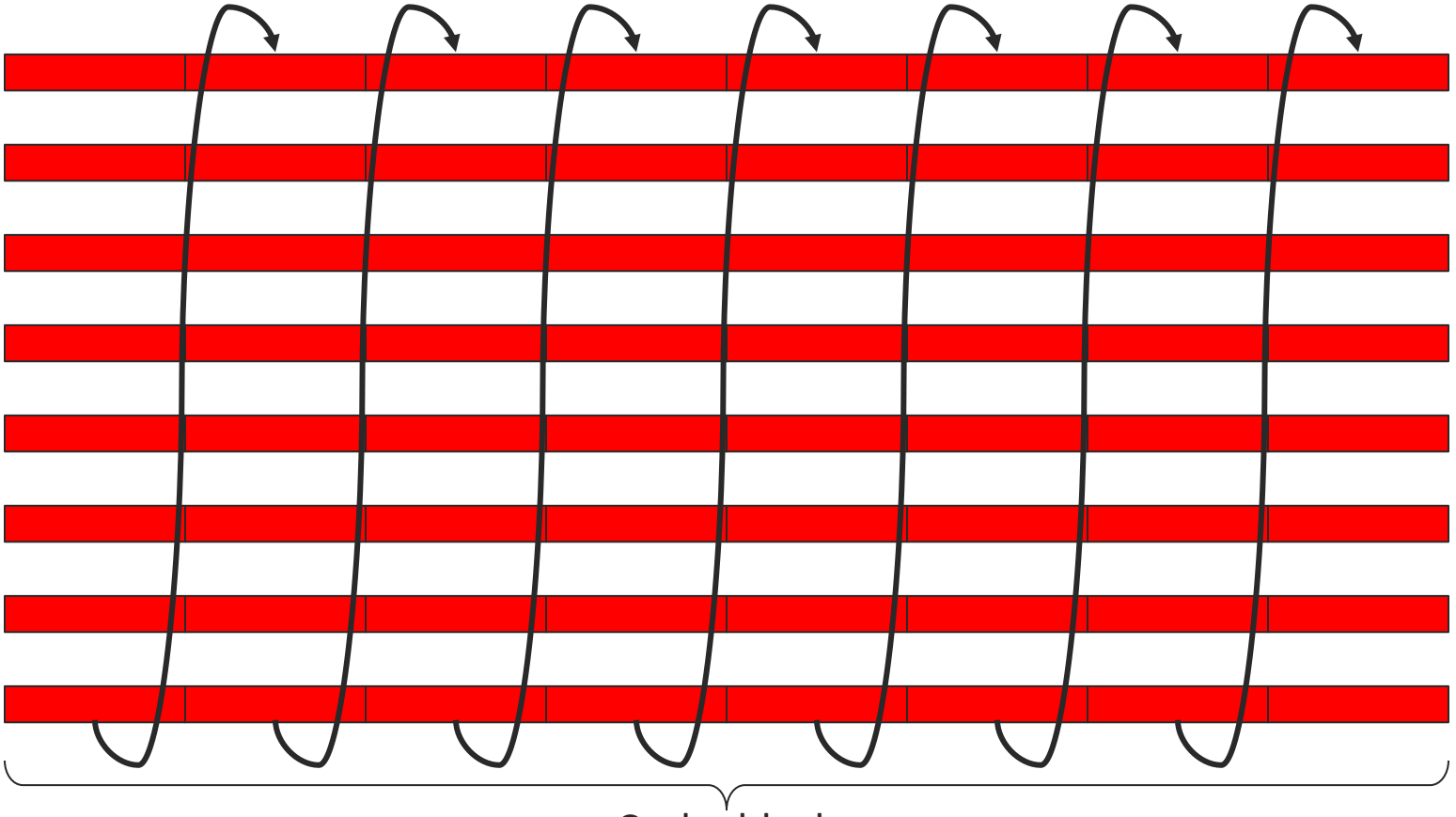
Row major storage example #1

- Hits and misses for A[0,0], A[0,1], … row by row access sequence; Hits = 7/8 = 87.5% (f_e)



Row major storage example #2

- Hits and misses for A[0,0], A[1,0], … column by column access and 4-block cache capacity; Hits = 0%



Lecture – Cache memory, part 2

My first book was so horrible I have deleted all copies of it. Thankfully, it was before the Internet, so there are no lurking caches of it anywhere.
– Andy Weir

Author of *The Martian*

How to improve cache performance?

- Read chapter 5
- Learning objectives
  - Remember the 3C's of cache misses
  - Explain preloading as a way to reduce compulsory misses
  - Explain the circuit for and operation of an associative cache
  - Explain the purpose of a replacement policy for an associative cache

Cache miss categories – the 3 C's

- Compulsory miss (also called cold-start miss or first-reference miss): name for the miss that must occur on the first access to a block of memory
- Capacity miss: occurs when the cache cannot simultaneously hold all blocks accessed by a program (see 90/10 Rule)
- Conflict miss: occurs when a set has fewer block storage locations that the number of blocks mapping to that set for a program's temporo-spatial locality (see Thrashing)
- 3 C's provide helpful guidance for cache designers seeking to improve performance by reducing misses

Reduce compulsory misses – 2 ways

- Static: Make cache block size larger reducing number of blocks used by a program

- Dynamic: While processor is busy making accesses that HIT within one block
  - Load the (possibly) known or the predicted next block into cache for the first time
  - If correct, compulsory miss of that block avoided

Reduce capacity misses – 2 ways

- HW: Increase cache capacity
  - Concerns are cost and heat
- SW: Reduce size of the machine program
  - Requires different target machine language, different source code, a smarter compiler, or some combination of these three
- SW: Reduce size of data structure temporal and/or spatial locality
  - Develop a better algorithm

Reduce conflict misses – a way

- Many blocks share the same index field
- For a direct mapped cache, if many blocks with same index are in demand by a program, can cause a series of miss/replacement cache actions as the program cycles its access requests among elements of this set of blocks
- Solution
  - Give each set k block storage locations
  - Zero conflict misses until number of in demand blocks at an index exceeds k

Four memory hierarchy design questions

- Q1: How is a block found?
- Q2: Where can a block be placed?
- Q3: Which block should be replaced on a miss?
- Q4: What happens when writing to memory?

- These questions help map out the design space for memory hierarchy
- Answers to these design questions affect hardware performance and inform software best practices

Q1. How is a block found?

- See Read Algorithm for direct mapped cache and generalize for Write to a block and Set Associative cache designs
- Corner case: Incorrectly finding a block
  - Shortly after Power On, cache is full of random bits that can falsely HIT
    - Set Valid bits = False before booting up
  - When a new process is launched by O/S
    - Set all Valid bits = False to prevent access of previous process' blocks
    - Or add Process ID field for each cache block

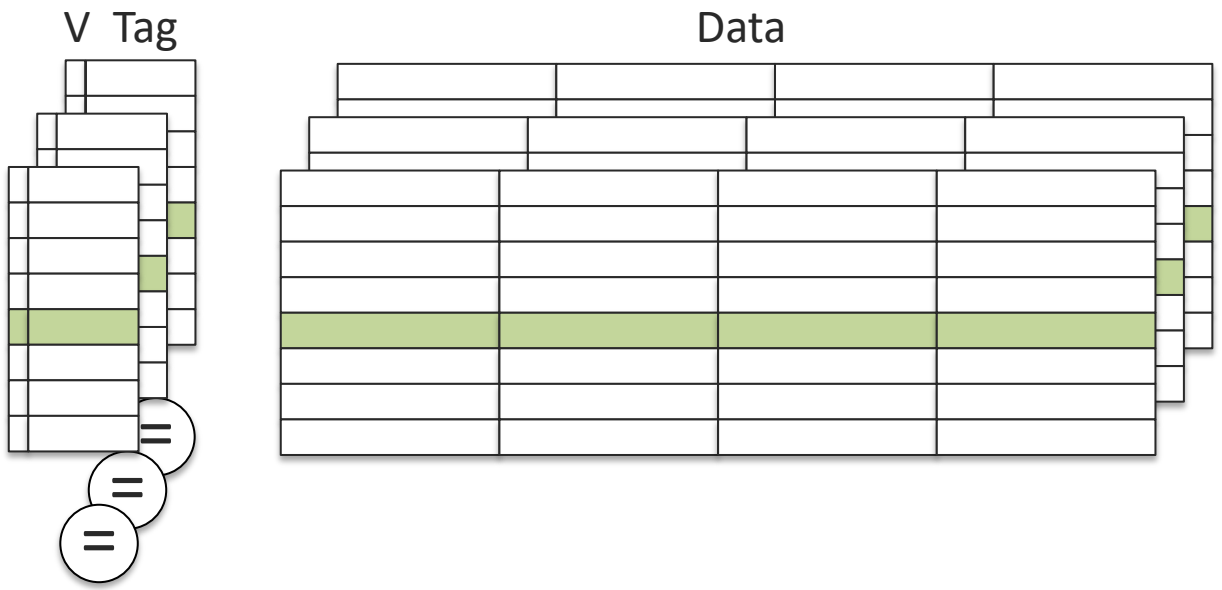
Q2. Where can a block be placed?

- If direct mapped cache, exactly one place

- If set associative cache, a block can be placed in any one of the k block storage locations provided by the set
  - k need not be a power of 2, but typically is

Set associative cache example

- Example, 3-way set associative design
- Replicate direct-mapped cache design 3 times
  - Search all block locations simultaneously for a tag match



Dimensions of cache design space

- Increase or decrease offset field size
- Increase/decrease sum of
  index field size + offset field size
- Increase/decrease cache associativity

Q3. Which block should be replaced on a miss?

- For direct mapped cache, set # of new block determines which block must be replaced
  - No choice, just point using set #, and replace the block in that set with the new block

Q3. Which block should be replaced? (cont.)

- Set associative cache stores k blocks per set
- When a new block is to be stored into a set, must choose which block in the set to replace with the new block
- A good choice can eliminate or postpone future conflict misses

Block replacement for associative cache

- To achieve acceptable speed, choice-making by hardware is required (Amdahl's Law again, a frequent action must be fast if overall performance is to be fast)

- What algorithm to make replacement choice?
- Optimal choice – replace that block in the set having its next use farthest into the future (infeasible)
  - Delays as much as possible incurring the repeat miss penalty to re-load that replaced block
  - Ideally, replace a block that is not used in the future
  - Optimal choice requires knowing future block accesses

Feasible replacement algorithms

- Least Recently Used (LRU)
  - When access a set, update block usage recency ranking
  - Problem: k blocks have k! rank orders
  - O(k!) hardware cost to track rank limits k to small values
  - When need to replace a block, choose the LRU block

- Not Recently Used (NRU)
  - Add "Recently Used" bit to each cache block
  - *Reset (clear) all Recently Used bits periodically*, then Set Recently Used bit each time a block is accessed (used)
  - When need to replace, choose the block with the lowest Keep Priority value using this table

Recently Used?	Keep Priority
Y	1
N	0

Don't forget the hardware for tie-breaking when more than one block has priority = 1

More HW algorithms for replacement

- First-in, first-out (FIFO)
- Pseudo-random
- Least frequently used (LFU)
- …

Thoughts on cache performance

- All replacement algorithms can work well at times, and all can fail miserably at times
  - Simulation on real workloads informs designer's choice
  - Cache-aware compiling can help deliver better performance
- Hardware + System software + App software determines achieved performance

Fully associative cache, no conflict miss

- Conflict miss category is eliminated if can place any block in any block storage location
  - Set # field is eliminated (zero bits for this field)
  - Search cache for a block using only the tag field
  - Expensive cache hardware: need a comparator for every tag in the cache, i.e., for every block storage location

When to use fully associative cache?

- **When conflict misses dominate**, AND
- ***When the miss penalty is enormous***
  - Example, DRAM and disk (hard or SSD) are adjacent memory hierarchy levels
    - SSD is much slower than DRAM, and hard disk is much-much slower
    - A DRAM miss incurs the enormous time penalty of an SSD or hard disk access
    - Further, SSD and hard disk are very, very, very slow compared to processor
    - A fully associative replacement policy implemented in SW and executed by a CPU can deliver a better replacement choice leading to a lower miss rate without significantly increasing the miss penalty
    - **Caution:** Need to ensure that there cannot be DRAM misses during the execution of the replacement policy software

What happens on a write to cache?

- Read chapter 5
- Learning objectives
  - Explain the write-through and write-back policies
  - Explain the dirty bit
  - Explain how Rule 3 procrastination increases pipeline throughput

Q4. What happens when writing to memory?

- Reading from the hierarchy is easy
  - Hierarchy Rules 1 & 2 move blocks up to faster levels, so CPU reads occur mostly within faster levels
- Writing is harder
  - Rule 1 demands that writes access the slowest level, a serious obstacle to performance if the CPU waits
  - Rule 3 (procrastination) is crucial to making writes faster

First write strategy – safety

- Write-through
  - CPU writes into the cache and the write immediately continues to the lowest level of the memory hierarchy
  - Satisfies Rule 3 of the Memory Hierarchy *as soon as possible; safe*
  - *Pay right now* the time cost to write to a slow level
  - Stalls the CPU unless there is a *HW write buffer*

  - Write-through to non-volatile storage is your policy when you Ctrl-S or ⌘-S

Procrastinate, to go faster. What !?!?

- Write-back policy for stores (procrastinate for Rule 3)
  - CPU writes a word into cache and marks its block as changed (dirty)
  - *Only when a dirty block must be replaced is the block written down one level*
  - Write-back strategy procrastinates on satisfying Rule 3, pushing into the future the cost of writing to a slower memory level
  - CPU sees writes complete at the speed of the L1 cache, i.e., speed similar to L1 read HITs
  - Amdahl's Law Bonus: Program may make multiple writes to one L1 cache block before referencing a memory address that forces the write-back action, thus *amortizing* the cost of one slow write-back operation across multiple, fast L1 cache write hits

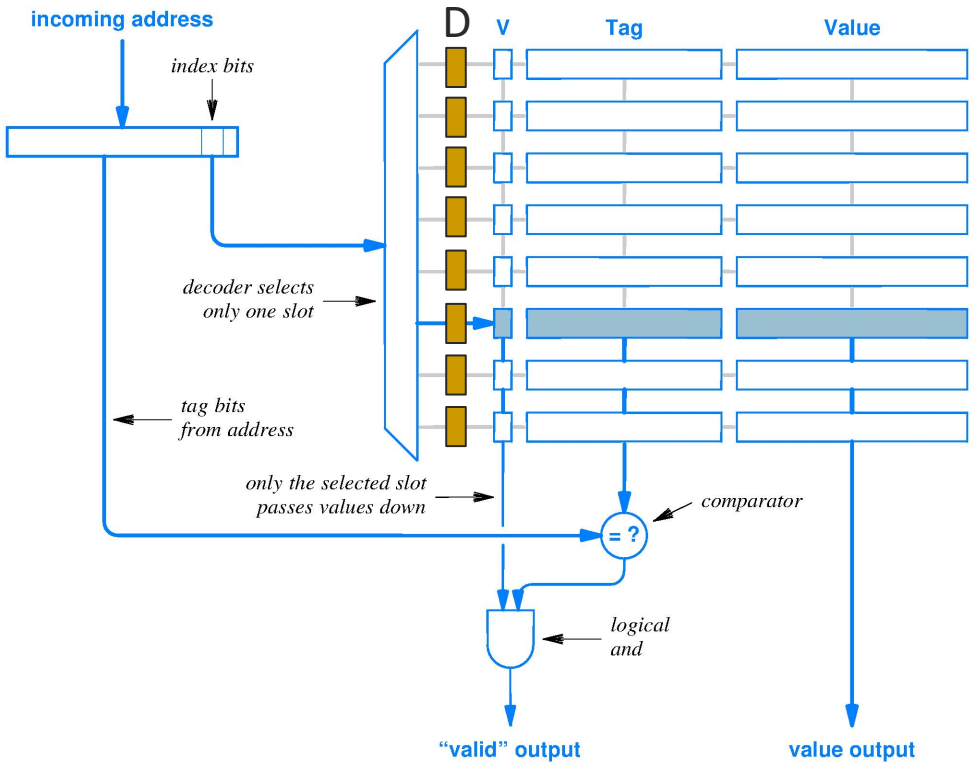
- Danger: If a write-back does not happen before the App crashes, then data is lost

Write-back algorithm

- Mark changed blocks for later write-back
  - Marking bit is called the dirty bit
- When write-back of a dirty block is forced by replacement policy decision
  - Write dirty block down to next lower memory level, preserving new information in dirty block (enforces first step of complying with memory hierarchy Rule 3)
  - Then new block may overwrite dirty block
    - Slower than replacing a “clean” block by overwriting

Write-back cache has dirty bit

- Dirty bit, D, asserts that a block is no longer a copy of a block at a lower level in the memory hierarchy
- Rule 3 says dirty block must be written down before being replaced



Write-back and NRU replacement

- Not Recently Used (NRU) with write-back
  - Revise table to prioritize keeping dirty blocks in cache
    - May postpone a write-back operation
  - Still, replace block with lowest Keep Priority value

Recently Used?	Dirty?	Keep Priority
Y	Y	3
Y	N	2
N	Y	1
N	N	0

Don’t forget tie-breaking hardware

Summary – the high-order bits on caching

- Read chapter 5
- Learning objectives
  - Gain an overview of the big issues of caching

Cache design space is rather large

- Increase/decrease offset field size
  - Increase/decrease words in block, so decrease/increase compulsory misses to extent accesses are sequential
  - Increase size of cache mux circuits to/from CPU
- Vary sum of `set_#_field_size + offset_field_size`
  - In/de cache capacity, de/in capacity misses & in/de cost
  - In/de cache circuit size which in/de access time
- Increase/decrease cache associativity
  - Decrease/increase conflict misses
  - In/de cache circuit size & in/de access time
- Vary replacement policy
  - vary # of conflict misses; vary miss penalty (perhaps)

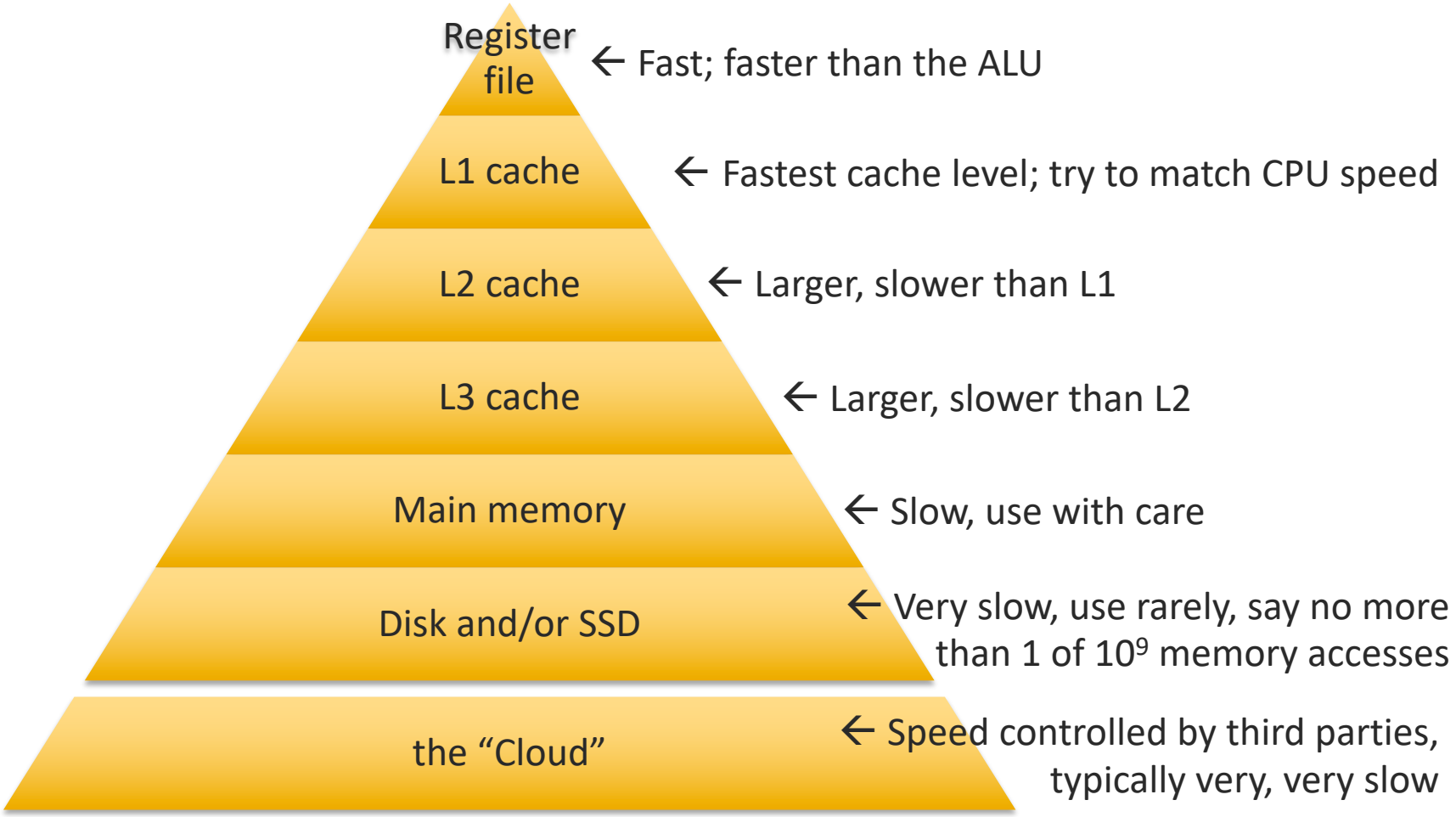
Summary

- Caching is a fundamental performance technique
- Very large design space
- To maximize caching benefit
  - Try to perform all operations on a data item before moving on to another data item (increase locality of references)
  - Move through multi-dimensional arrays in storage order
  - Use cache-aware compiling

Physical cache performance overview

- Read chapter 12.17, 12.25
- Learning objectives
  - Summarize cache performance and implications for programmers

Memory hierarchy – performance



Typical capacities and latencies

Level	Size in bytes	Typ. access time (ns)
64 64-bit Registers	512	0.1
L1 cache (64 KiB)	65,536	1
L2 cache (2 MeB)	2,097,152	3
L3 cache (8 MeB)	8,388,608	15
DRAM (8 GiB)	8,589,934,592	60
Hard disk (1 TB)	1,000,000,000,000	10,000,000
Archive (10 PB)	10,000,000,000,000,000	100,000,000,000

Number of digits is *logarithmic* with respect to magnitude
Nanoseconds have no *visceral* meaning
Let's put in more human terms and reconsider

Linear and human scales

Level	**if 1 byte = 1 mm Capacity as "Distance"**	**if 1 ns = 1 sec, One access takes**
64 64-bit (Register files, int and 754)	512 mm= 0.512 meters, from elbow to fingertips	0.1 second
64 KiB (L1, SRAM)	65.5 m (~2/3 length of a football pitch)	1 second
2 MeB (L2, SRAM)	2.1 km (Triple XXX to Airport Rd)	3 seconds
8 MeB (L3, SRAM)	8.4 km	15 seconds
8 GiB (DRAM)	8590 km (campus to Naples, Italy)	1 minute
1 TB (Hard disk)	2.5x distance to Moon	1 semester
10 PB (Tape)	0.001 light year (beyond Pluto)	3,169 years

Linear and human scale

Level	**if 1 ns = 1 sec, One access takes**
64 64-bit (Register files, int and 754)	0.1 second
64 KB (L1, SRAM)	1 second
2 MB (L2, SRAM)	3 seconds
8 MB (L3, SRAM)	15 seconds
8 GB (DRAM)	1 minute
1 TB (Hard disk)	1 semester
10 PB (Tape)	3,169 years

161,280 min/semester
Huge performance gap, far beyond gaps above
Special handing required

Lecture – Virtual memory

"Right now, I'm having amnesia
and déjà vu at the same time.
I think I've forgotten this before."

— Steven Wright

What is the motivation for virtual memory?

- Read chapter 13.1, 13.2, 13.8, 13.9, 13.13
- Learning objectives
  - Understand the motivations for virtual memory

Typical memory specs

Level	Size in bytes	Typ. access time (ns)
Register file, say 64 regs each 64 bits	512	0.25
DRAM (4 GB)	4,294,967,296	60.00
Hard disk (1 TB)	1,000,000,000,000	10,000,000.00

For performance reasons, application program use of DRAM and hard disk should be carefully managed.

Virtual memory motivations

- Provide a convenient memory address environment
- Allow efficient and safe [correct] sharing of memory among multiple programs
- Use main memory as a cache for hard disk content
- Speed some operating system tasks

- Note: motivations change with changes in memory technology

Provide convenient address space

- Do not know in advance which programs will share main memory
- Which programs are in main memory changes dynamically (i.e., all the time)
- Thus, we compile every program into its own *address space*, a separate range of addresses accessible only to this program

Provide safe sharing of memory

- Virtual memory (VM) system implements the translation of a program's address space to physical addresses used in main memory in the computer
- Translation enforces protection of a program's address space from other programs

Efficient use of main memory

- VM keeps in DRAM only the portion of address space currently in use by program, called the “working set” (program locality)
- Working set idea was created by Peter Denning, a former head of Purdue CS
- Safe sharing permits 1 copy of some information in main memory, rather than 1 copy/program
  - Shared Libraries – one copy shared by processes

Virtual memory speeds some O/S tasks

- Fork a process (gives the child process a copy of the memory of the parent process)
  - Virtual memory lets child process share parent's physical memory locations initially
  - Makes `fork` call fast; no need to immediately copy the entire parent environment for the child, sharply reducing the work needed to launch the child
  - Virtual memory provides "copy-on-write" capability to allow parent and child to diverge gradually

VM & cache in the memory hierarchy

- Cache hardware manages information movement between DRAM and CPU
- Virtual memory HW and SW manages info movement between disk and DRAM
- Virtual memory and cache concepts are the same, just very different size and time scales
- Differing historical roots of the two technologies lead to divergent terminology

VM & cache terminology relationships

- Virtual memory *block* is called a page
- Virtual memory *miss* is called a page fault
- Virtual memory mapping process
  - CPU emits a virtual address that is translated (mapped) by a combination of HW and SW to a physical address used to access main memory
  - Cache mapping process: done entirely with HW circuits [fixed at time of manufacture]

What is demand paged virtual memory?

- Read chapter 5
- Learning objectives
  - Explain the operation of a demand paged virtual memory system at the level of abstraction of our textbook

Virtual memory today: (Demand) Paging

- Unit of memory swapped is a fixed-size page
- Page size is optimized to disk of the day
  - Historic size: 512 bytes
  - Today size: 4 or 8 KBytes
- Program working set is well-approximated by loading an integer number of pages to main memory

Virtual memory performance issue

- Hard disk access latency is ~10^{-2} s; SSD access latency is ~10^{-5} s (both very, very slow)

- Goal: Never load a page into DRAM twice
  - Load a page >1 only if page was loaded then replaced
  - Thus, operate DRAM main memory as a fully associative cache *to eliminate conflict misses*

Fully associative main memory structure

- DRAM locations grouped into aligned page frames
- Page frame size in bytes = page size
- DRAM provides only raw storage locations; does not store valid bit, dirty bit, tag, etc. connected to data block, nor comparison circuits
- To search for a page in DRAM need *a software analogy to cache circuitry*, called the page table

Paging terminology

- Resident: a page that is currently in main memory
- Resident set: pages of a given application that are in main memory
  - Seek to have resident set trend toward the working set of the program

A paging design criteria set

- Keep only program working set in DRAM
- Choose page size to amortize disk/SSD latency
  - Amortize – to gradually achieve a value commensurate with an initial high cost
- Reduce page faults via excellent replacement choices
- Write-back
  - *Pages are very large compared to most data items in programs, so one dirty page can absorb many CPU store instructions to amortize the time of a one page write-back*
  - Huge disk/SSD access time makes write-through too slow

Paging design, Amdahl's Law issues

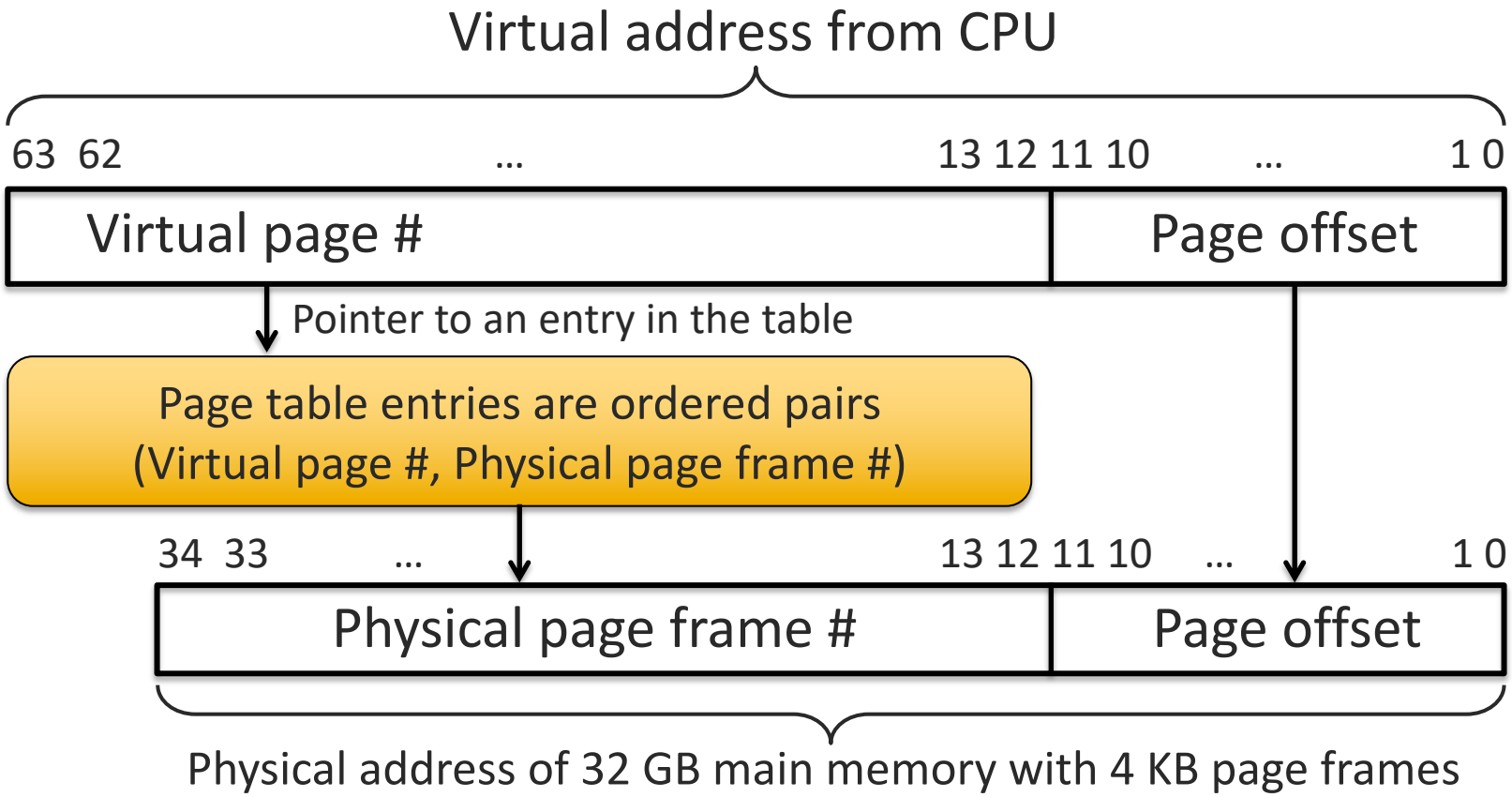
- Use hardware (fast, simple) for frequent events
  - Translate addresses from CPU (happens frequently) and detect pages not in main memory
- Use software (slow, but sophisticated) for infrequent events
  - Manage virtual memories of multiple processes
  - Configure data structures used by hardware
  - Make page replacement decisions intelligently

Mapping virtual addr. to physical addr.

Page size in bytes = $2^{\text{Offset\_field\_size\_in\_bits}}$

Page table maps virtual page to physical page frame in DRAM.

Aligning page frames in DRAM precludes need to map offset field.

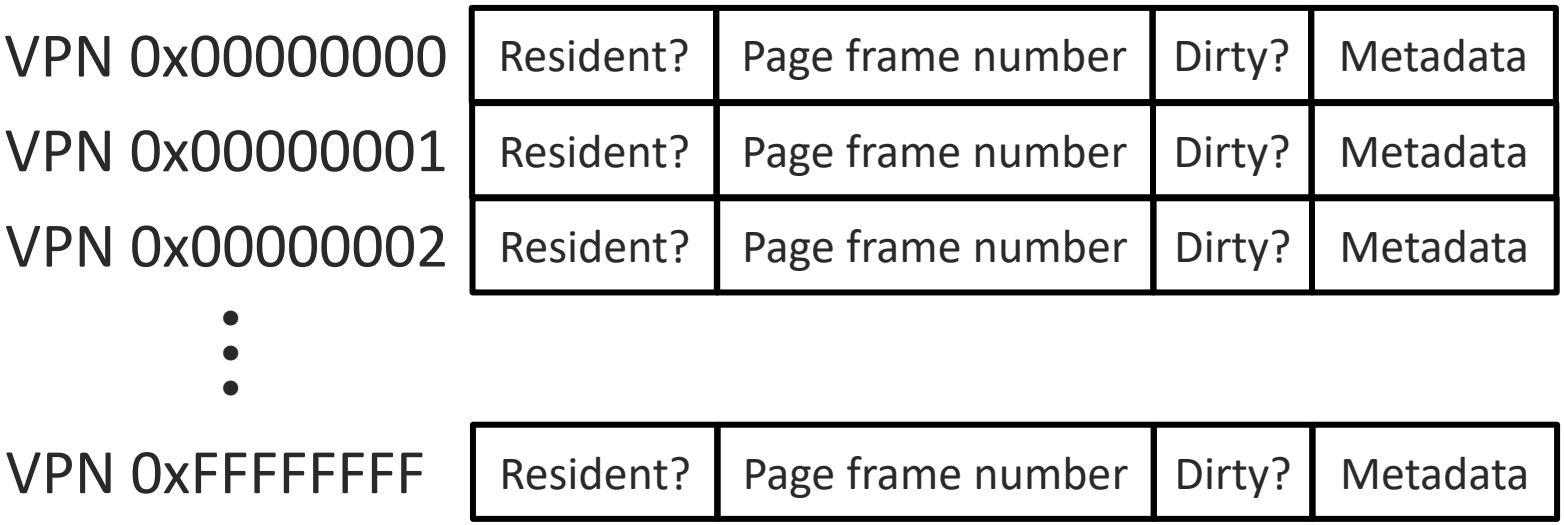


Page tables

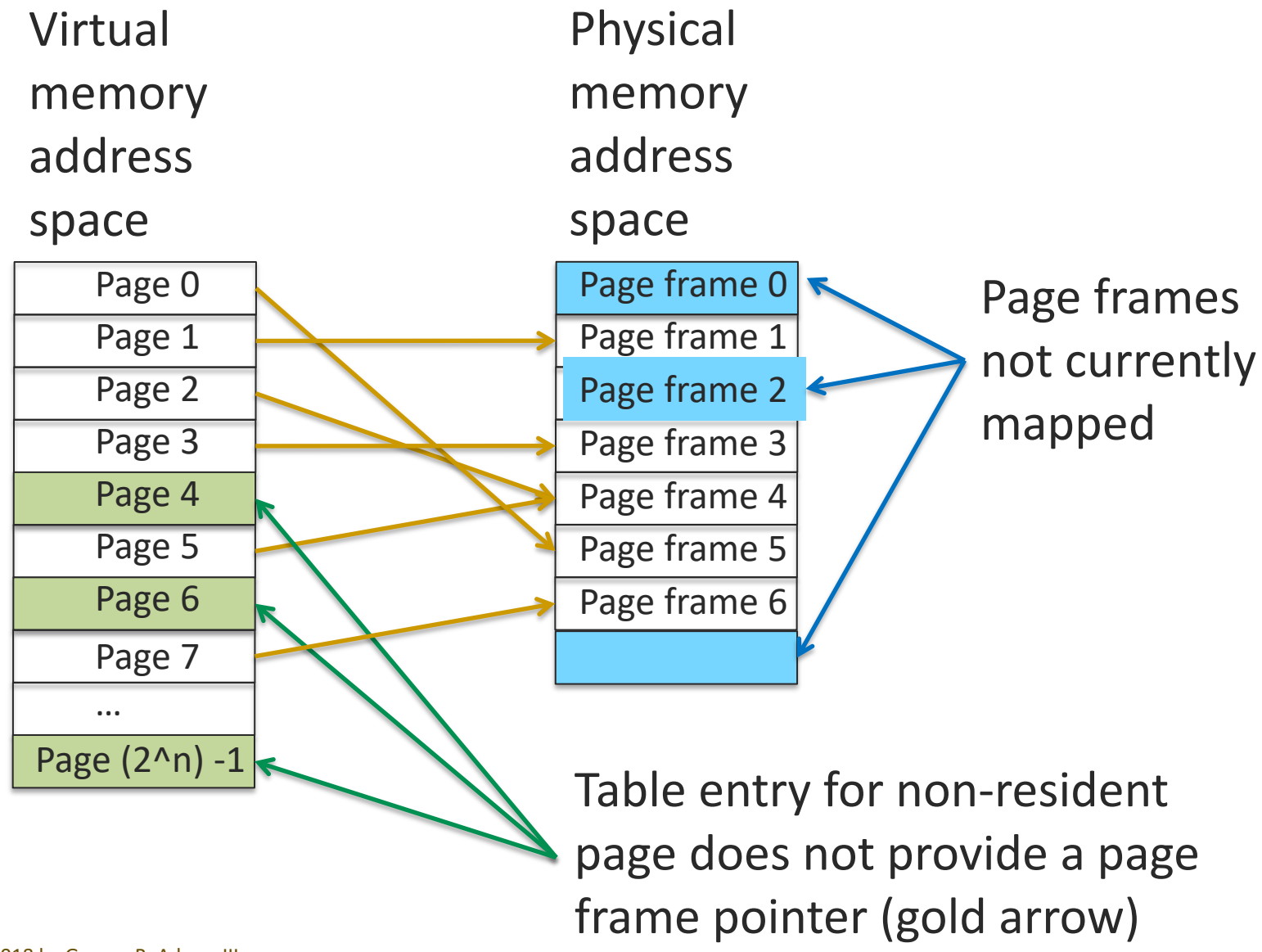
- Tables created and managed by O/S
- One page table per process; a 1-D array
  - Indexed by virtual page number, VPN
  - Table entry contains
    - Valid, or Resident, bit; false if page not in memory
    - Dirty bit
    - Physical memory page frame number corresponding to this virtual page number
    - Page metadata, a design choice, such as access permissions, data to support replacement algorithm

Page table diagram

- Page table array, pt[ ], element information is
  - Resident? and Dirty? bits
  - K-bit page frame number
  - J-bit page-describing metadata
- Lookup done as access of pt[0 + VPN]



Example address translation



Example page table (blue columns)

For 32-bit virtual addresses, 4096-byte pages, 24-bit physical memory address. Virtual address range and Virtual page # not part of actual Page Table (blue).

Virtual address range	Virtual page #	Mapped to	Page metadata
0x00000000 to 0x00000FFF	0x00000 = $(0)_{10}$	Page frame 0x024	**Text**. Read only, Executable
0x00001000 to 0x00001FFF	0x00001	Page frame 0xF05	**Data**. Read, Write, not Dirty
0x00002000 to 0x00002FFF	0x00002	Page frame 0xXXX	**Invalid (or not Resident)**
0x00003000 to 0x00003FFF	0x00003	Swap space (via inode)	**Swap**. Read, Write, not Dirty
…	…	…	…
0xFFFFE000 to 0xFFFFEFFF	0xFFFFE	Page frame 0xF43	Read, Write, not Dirty
0xFFFFF000 to 0xFFFFFFFF	0xFFFFF = $(2^{20}-1)_{10}$	Page frame 0x010	Read, Write, Dirty

Translate CPU *read* at location 0x00000124

- Page size = 4096 = 2^{12} bytes means 12-bit (3 hex digits) Offset field, so CPU virtual address field is 20-bit page number, 12-bit offset, thus parse 0x00000124 as 0x00000 = virtual page, 0x124 = byte offset within page.
- Main memory page frame size = 2^{12} bytes plus 24-bit byte addresses means page frame numbers are 12 bits. Lookup page frame in table, append offset yields hardware address. Use green row.
- Virtual address 0x00000124 from CPU translates to
  → (0x024 append 0x124) = 0x024124 main memory hardware address
  [gray column not a part of an actual page table]
- Page 0x00000 metadata allows read, so hardware performs CPU request

Virtual page #	Mapped to	Page metadata	Hardware address
0x00000 = $(0)_{10}$	Page frame 0x024	Read only, not Dirty, Executable (Text)	0x024XXX
0x00001	Page frame 0xF05	Read, write (Data), not Dirty	0xF05XXX
…	…	…	…
0xFFFFF = $(2^{20}\text{-}1)_{10}$	Page frame 0x010	Read, write, Dirty	0x010XXX

Translate CPU *write* at 0x00000124; SEGV

- As before:
- Virtual 0x00000124 → (0x024 append 0x124) = 0x024124 hardware
- However,
- Page 0x00000 metadata does NOT allow write into this page
- So memory hardware tells operating system about this memory access violation
- The operating system then forces this process to exit (terminate) with a SEGV (memory SEGmentation Violation) error

Virtual page #	Mapped to	Page metadata	Hardware address
$0x00000 = (0)_{10}$	Page frame 0x024	Read only, not Dirty, Executable (Text)	0x024XXX
0x00001	Page frame 0xF05	Read, write (Data), not Dirty	
…	…	…	
$0xFFFFF = (2^{20}\text{-}1)_{10}$	Page frame 0x010	Read, write, Dirty	

Translate CPU *read* at 0x00002000; Page fault

- Virtual address 0x00002000 → Invalid
- Page 0x00002 page metadata says Invalid; this is a "cache" MISS
- Memory hardware signals operating system of this miss, called page fault
- The operating system then executes code to (1) retrieve this page from the executable file on disk, (2) decide which memory page frame will hold the requested page, (3) if necessary, write-back replaced page, (4) write page 0x00002 into DRAM, (5) update page table entry, and (6) restart the process at the point where it reads at 0x00002000

Virtual page #	Mapped to	Page metadata	Hardware address
$0x00000 = (0)_{10}$	Page frame 0x024	Read only, not Dirty, Executable (Text)	0x024XXX
0x00001	Page frame 0xF05	Read, write (Data), not Dirty	0xF05XXX
0x00002	Page frame 0xXXX	not Resident	n/a
…	…	…	…

Summary

- Virtual memory analogous to caching
  - Fully associative placement
  - Replacement policy in software
- Supports memory access control, helps catch software bugs
- Supports having many processes share main memory yet be isolated from each other

What is cache coherence?

- Read chapter 5
- Learning objectives
  - Writing new information into the L1 level of the memory hierarchy causes a challenge for multi-core processors where each core has its own L1 cache level
  - Define the concept of cache coherence

Multiprocessor cache coherence

- Two processors, P1 and P2
  - Share main memory
  - Each has its own write-back L1 cache
- Value x is in neither cache
  - P1 has a cache miss when reading x, obtains x, stores a new x value into its write-back cache
  - Later, P2 has a cache miss when reading x, but needs to copy the missing block with x from P1 cache, not from main memory
- Example of the cache coherence problem for multiprocessors

Multicore CPUs and stale data

- Write back will lead to access of stale data (previous value) if each core has own cache and references same memory address

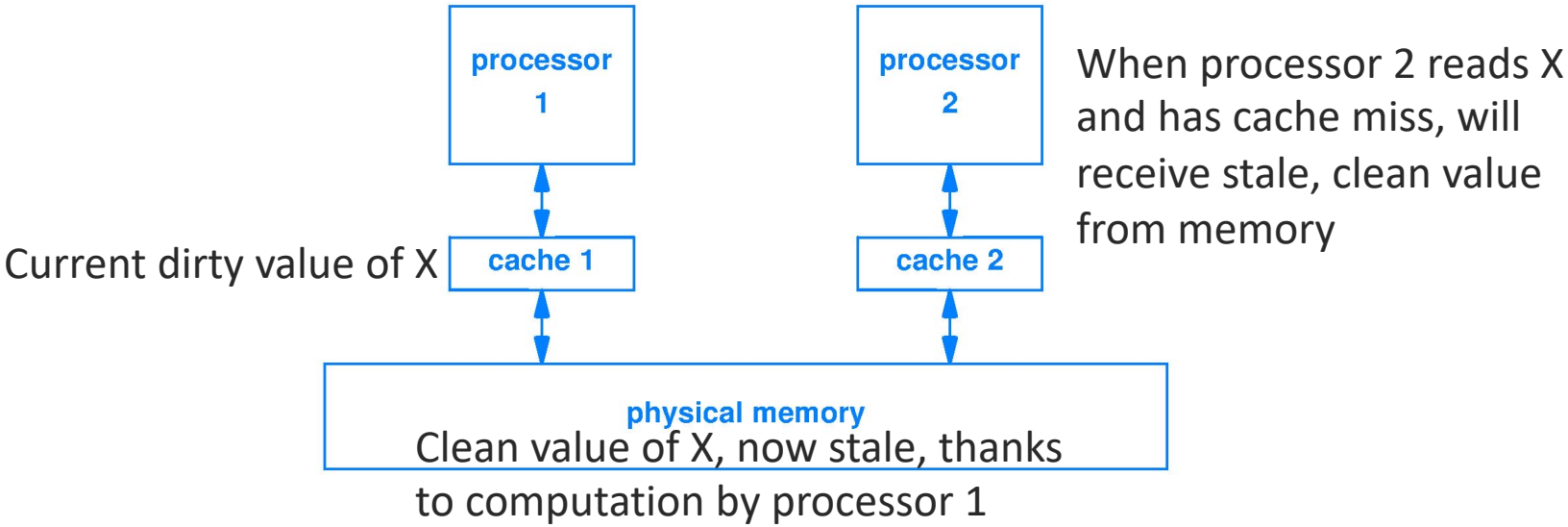
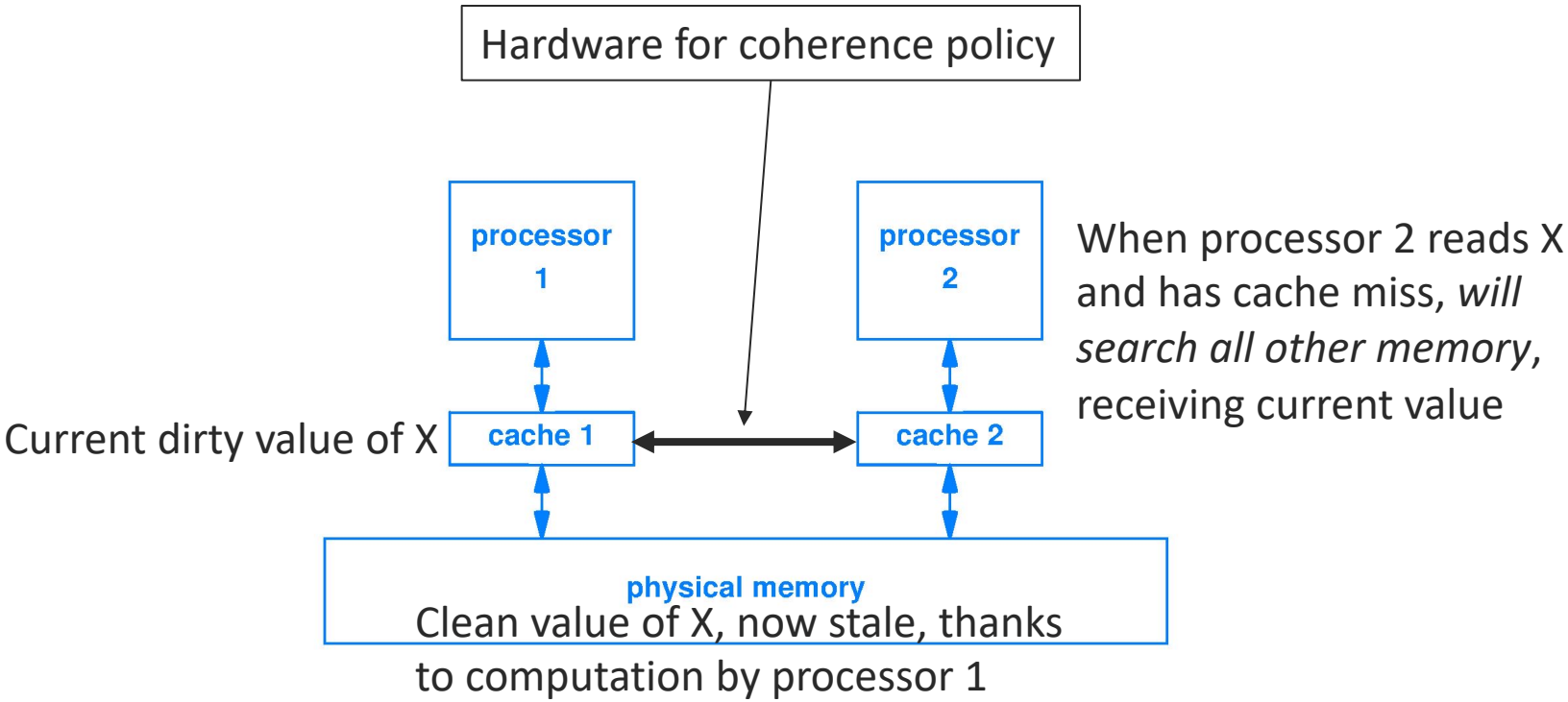


Figure 12.4 Illustration of two processors sharing an underlying memory. Because each processor has a separate cache, conflicts can occur if both processors reference the same memory address.

Cache coherence for multicore system

- Need a coherence policy, many possible
- One way, miss in cache 2 on X looks for X not only in memory but also in cache 1



Multiprocessor contention for resources

- Multiple processors means multiple, simultaneous use of caches, memory, I/O
- Example
  - If N processors share a memory bus
  - Then when 1 processor accesses memory, N-1 processors could be waiting for their own memory access, which is a substantial performance loss